



1

Building Classification Rules

- **Direct Method:**
 - Extract rules directly from data
 - e.g.: RIPPER, CN2, Holte's 1R
- **Indirect Method:**
 - Extract rules from other classification models (e.g. decision trees, neural networks, etc).
 - e.g: C4.5rules

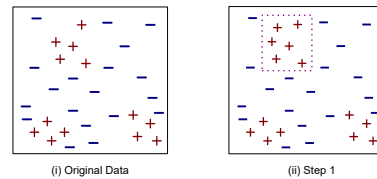
2

Direct Method: Sequential Covering

1. Start from an empty rule
2. Grow a rule using the Learn-One-Rule function
3. Remove training records covered by the rule
4. Repeat Step (2) and (3) until stopping criterion is met

3

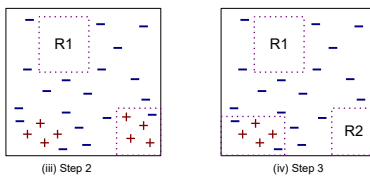
Example of Sequential Covering



A rule is **desirable** if it covers most of the positive and none or very few of the negative examples.

4

Example of Sequential Covering...



5

Sequential Covering Algorithm

- Inputs: labeled training dataset D
- Outputs: ruleset R that covers all instances in D
- Procedure:
 - Initialize R as the empty set
 - for each class C {
 - while D is nonempty {
 - Construct one rule r that correctly classifies some instances in D that belong to class C and does not incorrectly classify any non-C instances
 - Add rule r to ruleset R
 - Remove from D all instances correctly classified by r

6

Finding the next rule

- Notice that the sequential covering pseudocode does not specify how the rule r is to be constructed on a given pass through the inner while loop. Different sequential covering algorithms do this in different ways. The PRISM algorithm uses a depth-first search to construct the next rule for a given class C . Since the consequent of the rule must be the given class C , only the antecedent needs to be constructed; this is done by starting with an empty antecedent and iteratively adding the most promising attribute=value constraint next. The classification accuracy is used to rate candidate rules. This depth-first search continues until the resulting rule is specific enough that it makes no classification errors over the available data instances. Pseudocode follows below.

7

```

1) Inputs: nonempty dataset D, class C
2) Outputs: a single rule r with C as its consequent that covers some instances in D and that does not incorrectly
3) classify any non-C instances in D
4) Procedure:
5) Initialize r as a rule with an empty antecedent.
6) which predicts C without examining any attributes:
7)     => C
8) Initialize A as the set of all attributes over D
9)   while r incorrectly classifies some non-C instances of D {
10)     write r as ant(r) => C
11)     for each attribute-value pair (a=v), where a belongs to A and v is a
        value of a, compute the accuracy of the rule
12)         ant(r) and (a=v) => C
13)     let (a*=v*) be the attribute-value pair of maximum accuracy over D; in case of a tie, choose the pair that covers the greatest number of
        instances of D
14)     update r by adding (a*=v*) to its antecedent:
15)         r = ( ant(r) and (a*=v*) ) => C
16)     remove the attribute a* from the set A:
17)         A = A - {a*}
18)   }
19)   return r

```

8

8

Example

- % 9. Class Distribution:
- % 1. hard contact lenses: 4
- % 2. soft contact lenses: 5
- % 3. no contact lenses: 15
- @relation contact-lenses
- @attribute age {young, pre-presbyopic, presbyopic}
- @attribute spectacle-prescrip {myope, hypermetrope}
- @attribute astigmatism {no, yes}
- @attribute tear-prod-rate {reduced, normal}
- @attribute contact-lenses {soft, hard, none}
- @data
- %
- % 24 instances
- %
- r = (astigmatism=yes) and (tear-production-rate=normal) and (spectacle-prescription=myope) => hard

9

9

• young,	myope	no	reduced	none
• young	myope	no	normal	soft
• young	myope	yes	reduced	none
• young	myope	yes	normal	hard
• Young	hypermetrope	no	reduced	none
• young	hypermetrope	no	normal	soft
• young	hypermetrope	yes	reduced	none
• young	hypermetrope	yes	normal	hard
• pre-presbyopic	myope	no	reduced	none
• pre-presbyopic	myope	no	normal	soft
• pre-presbyopic	myope	yes	reduced	none
• pre-presbyopic	myope	yes	normal	hard
• pre-presbyopic	hypermetrope	no	reduced	none
• pre-presbyopic	hypermetrope	no	normal	soft
• pre-presbyopic	hypermetrope	yes	reduced	none
• pre-presbyopic	hypermetrope	yes	normal	none
• presbyopic	myope	no	reduced	none
• presbyopic	myope	no	normal	none
• presbyopic	myope	yes	reduced	none
• presbyopic	myope	yes	normal	hard
• presbyopic	hypermetrope	no	reduced	none
• presbyopic	hypermetrope	no	normal	soft
• presbyopic	hypermetrope	yes	reduced	none
• presbyopic	hypermetrope	yes	normal	none

10

10

- Age = young : 2/8

- Total 12 values

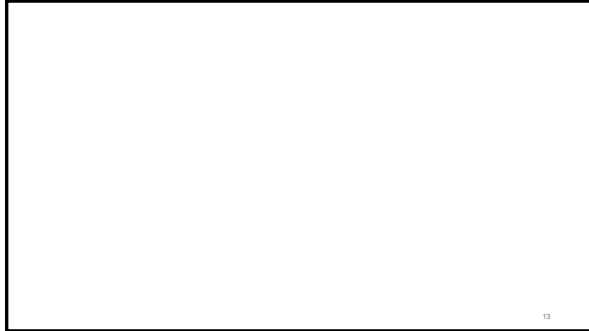
•

11

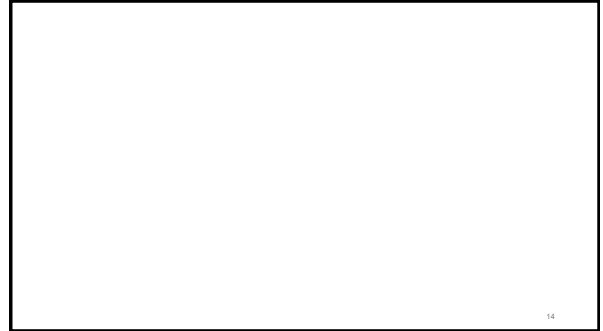
11

12

12



13



14



15

Aspects of Sequential Covering

- Learn-One-Rule function is to extract a classification rule that covers many of the positive examples and none/few of the negative examples.
- It is computationally expensive given the exponential size of the search space.
- The function grow the rules in a greedy fashion
- Instance Elimination
- Rule Evaluation
- Stopping Criterion
- Rule Pruning

16

Rule Evaluation

• Metrics:

$$u_{\text{Accuracy}} = \frac{n}{n}$$

• Laplace

$$= \frac{n_c + 1}{n + k}$$

• M-estimate

$$= \frac{n_c + kp}{n + k}$$

Laplace and m-estimate are equivalent if $p=1/k$

Rule r1, covers 50 positive examples and 5 negative examples

Rule r2, covers 2 positive examples and 0 negative examples

n : Total number of instances
 n_c : Number of instances of class c covered by rule
 k : Number of classes
 p : Prior probability

17

Rule evaluation: example

Suppose we have a 2-class problem with 60 positive and 100 negative examples. Consider

1. R_1 covers 50 positive examples and 5 negative.
2. R_2 covers 2 positive examples and no negative.

Accuracy of R_1 is $50/55 = 0.91$. Accuracy of R_2 is $2/2 = 1$.

With Laplace correction this becomes:

R_1 : $(50+1)/(55+2) = 0.89$ R_2 : $(2+1)/(2+2) = 0.75$

With m-estimate ($p=60/160=3/8$):

R_1 : $(50+2 \times 3/8)/(55+2) = 0.89$ R_2 : $(2+2 \times 3/8)/(2+2) = 0.69$

18

- but it is not the better rule because of its small coverage.
- From this example, we see that accuracy on its own is not a reliable estimate of rule quality.
- Coverage on its own is not useful either or a given class we could have a rule that covers many tuples, most of which belong to other classes!
- Thus, we seek other measures for evaluating rule quality, which may integrate aspects of accuracy and coverage.

19

19

Stopping Criterion and Rule Pruning

- Stopping criterion
 - Compute the gain
 - If gain is not significant, discard the new rule
- Rule Pruning
 - Similar to post-pruning of decision trees
 - Reduced Error Pruning:
 - Remove one of the conjuncts in the rule
 - Compare error rate on validation set before and after pruning
 - If error improves, prune the conjunct

20

Summary of Direct Method

- Grow a single rule
- Remove Instances from rule
- Prune the rule (if necessary)
- Add rule to Current Rule Set
- Repeat

21

Direct Method: RIPPER

- For 2-class problem, choose one of the classes as positive class, and the other as negative class
 - Learn rules for positive class
 - Negative class will be default class
- For multi-class problem
 - Order the classes according to increasing class prevalence (fraction of instances that belong to a particular class)
 - Learn the rule set for smallest class first, treat the rest as negative class
 - Repeat with next smallest class as positive class

22

Direct Method: RIPPER

- Growing a rule:
 - Start from empty rule: $\phi \rightarrow \text{pos}$ or $\phi \rightarrow \text{class}$
 - Add conjuncts as long as they improve FOIL's information gain

$$\text{FOIL_Gain} = \text{pos}' \times \left(\log_2 \frac{\text{pos}'}{\text{pos}' + \text{neg}'} - \log_2 \frac{\text{pos}}{\text{pos} + \text{neg}} \right)$$

Where pos and neg are number of positive and negative instances covered by the initial rule R_0 , and pos' and neg' are number of positive and negative instances covered by the new rule R_1 .

- Stop when rule no longer covers negative examples

23

Rule growing: Ripper

Suppose R_0 covers 60 positive cases and 40 negative.

We compare two possible refinements of R_0 :

1. R_1 covers 50 positive and 10 negative cases.
2. R_2 covers 30 positive and 5 negative cases.

Their information gains are:

$$\text{Gain}(R_0, R_1) = 50 \left(\log_2 \frac{5/6}{3/5} \right) \approx 23.7$$

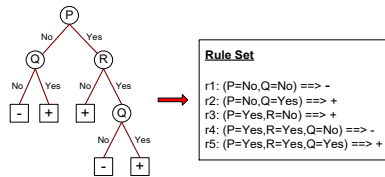
$$\text{Gain}(R_0, R_2) = 30 \left(\log_2 \frac{6/7}{3/5} \right) \approx 15.4$$

Hence we prefer refinement R_1 even though it has lower accuracy.

24

24

Indirect Methods



Look at rule 2, 3, 5

Can be simplified into
 R2': Q= yes → +
 R3: P=yes ^ R=no → +

25

Rule Pruning

- A rule is pruned by removing a conjunct (attribute test). We choose to prune a rule, R , if the pruned version of R has greater quality, as assessed on an independent set of tuples.
- Given a rule, R ,

$$FOIL_Prune(R) = \frac{pos - neg}{pos + neg}$$

if the *FOIL Prune* value is higher for the pruned version of R , then we prune R .

26

26

Indirect Method: C4.5rules

- Extract rules from an unpruned decision tree
- For each rule, $r: A \rightarrow y$,
 - consider an alternative rule $r': A' \rightarrow y$ where A' is obtained by removing one of the conjuncts in A
 - Compare the error rate for r against all r 's
 - Prune if one of the r 's has lower error rate
 - Repeat until we can no longer improve generalization error

27

Advantages of Rule-Based Classifiers

- As highly expressive as decision trees
- Easy to interpret
- Easy to generate
- Can classify new instances rapidly
- Performance comparable to decision trees

28